V-Decent Application Development Guide (en, V2_3)

Document Version: 2.3
Target Platform: V-Decent Distributed Node Cluster (Coolify Orchestration)
Line Spacing: 1.15

1. Introduction

This document serves as the standardized engineering guide for developers building, packaging, and deploying multi-tier applications across the V-Decent distributed network infrastructure. By optimizing container networking topologies, the platform ensures secure, zero-trust host execution alongside high density, scaling up to 20 production application stacks per bare-metal or cloud host instance without system resource starvation.

2. Development Lifecycle & Responsibilities

Application engineering teams maintain complete ownership over internal software design, local runtime composition, and environment isolation logic. Platform operations teams manage high-availability host boundaries, global load balancing, and backing storage networks. Handover into staging or production systems is driven automatically via declarative configuration manifests managed by Coolify control panels.

3. Supported Application Types

- **Stateless Compute Services:** API gateways, frontend web applications, background worker microservices, and chronological task processors.
- **Stateful Storage Services:** Relational and non-relational database instances, persistent data caching nodes, and structured file queues utilizing platform-managed underlying storage volumes.

4. Technical Requirements for V-Decent Compatibility

To operate seamlessly inside the V-Decent multi-tenant environment, all applications must completely decouple public routing infrastructure from internal workload communication. The underlying Docker daemon dynamically manages subnet pools behind the scenes. Developers must follow standard, non-intrusive container formatting specifications to avoid proxy routing faults and upstream communication drops.

4.1 Docker Configuration Specification

The following parameters are mandatory for any application stack submitted for platform deployment. Compliance ensures that individual containers are correctly registered by automated proxy components while maintaining zero host-level port exposures.

Requirement	Technical Specification & Description
No Host Port Mapping	Do not map, expose, or bind physical port configurations directly to the host network interface (e.g., avoid ports: "8080:80"). Direct bindings create network conflicts across multi-tenant nodes. All inbound ingress routing paths are dynamically managed and isolated by Coolify proxy instances.
Port Declaration	Use the expose array parameter to identify internal networking endpoints. This alerts upstream container bridges where application traffic is listening.
Native Network Isolation	Define internal-only software bridge networks (e.g., external-tier, internal-tier) within the application manifest. **Do not** hardcode reference configurations to global external networks like vdecent-ingress. This prevents conflicts with runtime orchestration engines.
Environment Variables	Isolate all secrets, connection tokens, and environment parameters into standard .env configurations. Include a fully documented, unpopulated .env.example reference profile inside the root repository.
Liveness & Readiness Probes	Integrate explicit Docker engine healthcheck routines for all downstream data storage systems and caching daemons to confirm initialization status before compute workers begin boot cycles.

4.2 Reference Implementation Setup

The following standardized blueprint demonstrates a highly resilient, two-tier architecture comprising a web runtime environment and an isolated database layer. This structure utilizes isolated application bridges that draw subnets automatically from expanded host pools while allowing Coolify to cleanly map its public ingress hooks at runtime.

```
services:
  app:
```

```
    build: .
    restart: always
    environment:
      - DATABASE_URL=postgres://user:password@db:5432/activitylog
    expose:
      - "80"
    depends_on:
      db:
        condition: service_healthy
    networks:
      - external-tier  # Internal web-tier isolation
    labels:
      - "coolify.managed=true"

  db:
    image: postgres:16-alpine
    restart: always
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=password
      - POSTGRES_DB=activitylog
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d activitylog"]
      interval: 5s
      timeout: 5s
      retries: 10
    networks:
      - internal-tier  # Database remains completely isolated from web
vectors

networks:
  external-tier:      # Dynamically allocated bridge network for web
ingress
    driver: bridge
  internal-tier:       # Dynamically allocated bridge network for
isolated database traffic
    driver: bridge

volumes:
  postgres_data:
```

5. Reference Implementations & Handover Workflows

When an application repository is integrated via Coolify, the system parses the

docker-compose.yaml file, spins up the declared external-tier and internal-tier bridges, and automatically injects its native, managed ingress network path directly into the service container marked with the coolify.managed=true label. This guarantees that Traefik or Caddy proxies maintain a direct, clean network route to the web server endpoint, completely eliminating the risk of 503 Service Unavailable errors caused by structural network mismatches.